

Cyberpunk Velocity

Integrantes

Iván Sardón Medina
José Miguel Valdivia
Rodrigo Silva Murillo

Curso: Computación Gráfica

Departamento: Departamento de Computación

Universidad: Universidad Católica San Pablo

E-mails:

Iván Sardón Medina: ivan.sardon@ucsp.edu.pe
José Miguel Valdivia: jose.valdivia.castillo@ucsp.edu.pe
Rodrigo Silva Murillo: rodrigo.silva@ucsp.edu.pe

Junio de 2026

1 Introducción y motivación del proyecto

Cyberpunk Velocity es una escena 3D interactiva desarrollada con OpenGL 3.3 Core Profile. El programa muestra un Porsche en una carretera urbana nocturna con edificios, vegetación, postes de luz, semáforo, cielo sintético, sol con halo, niebla y varias cámaras. La motivación del proyecto fue integrar en una sola aplicación los temas centrales del curso: transformaciones 3D, proyección perspectiva, cámaras lookAt, carga de mallas OBJ, texturizado, iluminación, blending y animación temporal.

La escena fue diseñada con una estética cyberpunk: tonos oscuros, luces magenta/cian, brillos aditivos, ventanas iluminadas y niebla azulada. Esta dirección visual obligó a resolver problemas técnicos concretos, como el orden de render para transparencias, el manejo de luces múltiples, la mezcla de objetos importados y geometría generada por código, y la sincronización entre semáforo, frenado y desplazamiento de la ciudad.

El programa se apoya en cabeceras propias que funcionan como un pequeño motor gráfico: `draw2D.h`, `draw3D.h`, `lighting.h`, `transformations.h`, `camera.h`, `CameraSystem.h`, `objetos.h`, `Porsche.h` y `Sky.h`. Estas librerías encapsulan matrices, shaders, VAO/VBO/EBO, carga OBJ, texturas, materiales, luces, grupos de formas, cámaras y objetos compuestos.

2 Descripción del Proyecto Final de Programación

El archivo `main.cpp` coordina el flujo principal. Primero inicializa GLFW, solicita un contexto OpenGL 3.3 Core Profile, crea la ventana, carga GLAD, activa `GL_DEPTH_TEST` y desactiva `GL_CULL_FACE`. Después construye los tres elementos principales de la escena: `Objetos::BloqueCiudad`, `PorscheCar` y `Sky`. Finalmente configura el sistema de cámaras y entra al loop de render.

```
Inicializar GLFW y crear ventana
Cargar GLAD y activar depth test
Cargar ciudad, Porsche y cielo
Configurar sistema de camaras

while ventana abierta:
    calcular deltaTime con glfwGetTime()
    procesar input
    actualizar animacion, ruedas y frenado
    limpiar y registrar luces
    subir proyeccion y vista
    dibujar cielo, ciudad y Porsche
    intercambiar buffers
```

El render usa matrices globales administradas desde `draw2D.h`. Antes de dibujar se llama a `setProjectionMatrix(perspective(...))` y `setViewMatrix(activeViewMatrix(cameras))`. Todas las formas que usan los shaders por defecto leen esas matrices, por lo que el cielo, la ciudad y el auto quedan bajo la misma cámara y proyección.

La estructura lógica del programa es:

```
Cyberpunk Velocity/
main.cpp           // loop principal e inicializacion
draw2D.h / draw3D.h // base de dibujo, VAO/VBO, formas y OBJ
lighting.h         // luces, materiales y shader Blinn-Phong
transformations.h  // Mat4, Transform y composicion TRS
camera.h           // lookAt, perspective y camara libre
CameraSystem.h     // modos de camara e input
objetos.h          // ciudad, carretera, postes, semaforo
Porsche.h          // vehiculo, piezas, texturas y ruedas
```

Sky.h	// domo de cielo, sol y halo
Models/	// OBJ, MTL y texturas PNG

El proyecto separa responsabilidades por archivo. `transformations.h` contiene `Mat4`, `Transform`, traslaciones, rotaciones, escalas y multiplicación de matrices. `draw2D.h` define `Shape`, `ShapeGroup`, `Shader`, `Texture2D`, carga de texturas y blending. `draw3D.h` agrega `GenShape` para OBJ, `Sphere`, `GlowHalo` y `GlowCone`. `lighting.h` define materiales, luces y el shader Blinn-Phong. `CameraSystem.h` concentra la interacción de cámara. `objetos.h`, `Porsche.h` y `Sky.h` construyen los elementos visuales de alto nivel.

Las funciones de soporte más importantes son las siguientes:

Función o clase	Responsabilidad en el proyecto
<code>setProjectionMatrix</code> y <code>setViewMatrix</code>	Guardan matrices globales usadas por los shaders por defecto. Antes de dibujar se actualiza la proyección perspectiva y la vista activa para que todos los objetos compartan la misma cámara.
<code>Shape::draw</code>	Selecciona shader, sube uniformes, enlaza el VAO y ejecuta <code>glDrawElements</code> o <code>glDrawArrays</code> . Si hay normales y material iluminado, usa el shader Blinn-Phong.
<code>ShapeGroup::draw</code>	Multiplica la matriz del grupo por la matriz local de cada hijo. Se usa para agrupar piezas del Porsche y aplicar transformaciones comunes.
<code>GenShape(objPath)</code>	Carga modelos OBJ de carretera, edificios, Porsche y decoración. Convierte el archivo a vértices intercalados con posición, color, normal y UV.
<code>loadTexture2D</code>	Carga PNG mediante <code>stb_image</code> , crea <code>GL_TEXTURE_2D</code> , define filtros, wrapping y formato interno.
<code>enableLighting(material)</code>	Asocia un <code>Material</code> a una malla y activa el shader iluminado. Une geometría, textura y luces en el render final.
<code>clearLights</code> y <code>addSpotLight</code>	Reconstruyen el arreglo de luces cada frame para evitar acumulación y mantener coherencia con objetos animados.
<code>BloqueCiudad::setLayerOffsets</code>	Desplaza las capas de ciudad en Z, base de la carretera infinita y del parallax entre objetos cercanos y fondo.

El orden de render también es parte de la arquitectura. Primero se dibuja el cielo sin depth test porque representa el fondo. Luego se dibuja la ciudad opaca y sus emisores luminosos. Finalmente se dibuja el Porsche, dejando los cristales para el final con `glDepthMask(GL_FALSE)`. Este orden evita que objetos transparentes escriban profundidad como si fueran sólidos y reduce artefactos visuales.

3 Funcionalidades de su programa

Keyboard y movimiento de cámara

El input se procesa en `processInput`. La tecla `ESC` cierra la ventana y `SPACE` pausa o reanuda la animación. Las teclas `1`, `2`, `3`, `4`, `5` y `L` cambian entre cámara general, conductor, superior, lateral izquierda, lateral derecha y cámara libre. Para evitar cambios repetidos mientras una tecla queda presionada, `pressedOnce` detecta el flanco de pulsación.

Las cámaras fijas se calculan con `activeViewMatrix`. La función usa `lookAt`, que construye una matriz de vista con tres vectores: dirección frontal normalizada, eje lateral por producto cruz y eje vertical corregido. Las cámaras general y de conductor no se desplazan libremente; solo modifican yaw/pitch dentro de límites para mantener una vista controlada. La cámara libre usa `Camera::rotateFromMouse`,

movimiento con WASD y desplazamiento vertical con Q/E. El scroll modifica el FOV dentro de un rango controlado.

Entrada	Acción
ESC	Cierra la ventana principal mediante <code>glfwSetWindowShouldClose</code> .
SPACE	Alterna pausa/reanudación de la animación usando detección de flanco.
1 a 5	Cambian entre cámara general, conductor, superior, lateral izquierda y lateral derecha.
L	Activa la cámara libre. En este modo el mouse se recentra para calcular desplazamientos relativos.
W/A/S/D, flechas	Ajustan orientación en cámaras limitadas o desplazan la cámara libre.
Q/E	Bajan/suben verticalmente en la cámara libre.
Scroll	Modifica el FOV dentro de límites para evitar zoom excesivo o deformación extrema.

Transformaciones y animación

Las matrices usan formato column-major compatible con OpenGL. La composición principal es:

```
T * Rz * Ry * Rx * S
```

Esto aplica escala local, luego rotación y al final traslación. En `objetos.h`, `makeTransform` centraliza la conversión desde Blender: posición OpenGL como (X, Z, -Y), escala como (ScaleX, ScaleZ, ScaleY) y rotaciones adaptadas al sistema de ejes del proyecto.

La animación no mueve el Porsche hacia adelante; mueve el mundo alrededor de él. `aniBloq1` desplaza la capa cercana y `aniBloq2` desplaza la capa media a una fracción de la velocidad. Esto produce parallax y simula movimiento continuo. Cuando una capa supera el límite, vuelve a una posición anterior para que la carretera parezca repetirse. La rotación de ruedas se calcula en `PorscheCar::update`:

```
wheelRotation += speed * deltaTime * wheelRotationFactor;
```

El semáforo se sincroniza con `animationTime`. `TrafficLightObjects::signalState` evalúa un ciclo de 10 segundos: verde de 0 a 5, amarillo de 5 a 7 y rojo de 7 a 10. En `main.cpp`, si la luz no está verde y el semáforo queda delante del vehículo dentro de una distancia corta, `speedMultiplier` tiende a cero. Cuando la condición desaparece, vuelve progresivamente a uno, evitando cambios instantáneos de velocidad.

Iluminación

La iluminación se basa en Blinn-Phong. Cada `Material` contiene ambiente, difuso, especular, emisivo, brillo, opacidad, recorte alfa y textura difusa. El shader calcula ambiente global, componente difusa, reflejo especular, emisividad, atenuación por distancia, transición suave en spotlights y niebla.

En cada frame se llama a `clearLights` y luego se vuelven a registrar las luces. La escena usa ambiente global (0.20, 0.20, 0.22), niebla azulada, dos luces direccionales neon y spotlights en los postes. Los postes tienen una doble representación: `addSpotLight` crea la luz real que afecta el shader, mientras `drawEmitters` dibuja un panel brillante y un `GlowCone` aditivo para mostrar el haz.

El vertex shader iluminado transforma la posición local con `uModel` y calcula la normal con `mat3(transpose(inverse(uModel)))`. Esto evita que escalas no uniformes deformen las normales. En el fragment shader, la luz direccional usa `-direction`; las luces puntuales y spotlights calculan el vector desde el fragmento hacia la luz. La especularidad usa el vector medio $H = \text{normalize}(L + V)$, propio de Blinn-Phong.

Elemento	Configuración visual
Pintura del Porsche	Material con especularidad alta y <code>shininess = 96.0</code> , pensado para reflejo fuerte en la carrocería.
Cristales	Opacidad parcial, especularidad alta, alpha blending y escritura de profundidad desactivada.
Hojas de árboles	Texturas RGBA con <code>alphaCutoff</code> ; se descartan píxeles transparentes para evitar planos rectangulares visibles.
Postes, sol y ventanas	Emisividad o blending aditivo para producir brillo visible en la escena nocturna.

Modelos y librerías para cargar modelos

La carga de modelos se realiza con **GenShape**. Esta clase lee OBJ con posiciones `v`, coordenadas UV `vt`, normales `vn` y caras `f`. Las caras de más de tres vértices se triangulan; si falta una normal, se genera con producto cruz. La geometría se sube al GPU con atributos intercalados:

```
layout(location = 0) vec3 aPos;
layout(location = 1) vec4 aColor;
layout(location = 2) vec3 aNormal;
layout(location = 3) vec2 aTexCoord;
```

Las texturas se cargan con `loadTexture2D`, que usa `stb_image`. Las opciones permiten invertir verticalmente la imagen, usar formato `sRGB`, controlar mipmaps y escoger entre `GL_REPEAT` o `GL_CLAMP_TO_EDGE`. Aunque existen archivos `.mtl`, el cargador propio no interpreta materiales MTL; por eso el aspecto visual se define manualmente con `Material` y `enableLighting(...)`.

El formato final de vértice para mallas iluminadas ocupa 12 flotantes. La posición local define la geometría, el color por vértice permite tintes simples, la normal alimenta el cálculo de iluminación y las UV permiten muestrear mapas difusos:

```
layout(location = 0) vec3 aPos;
layout(location = 1) vec4 aColor;
layout(location = 2) vec3 aNormal;
layout(location = 3) vec2 aTexCoord;
```

Esta organización permite que primitivas generadas por código y modelos importados pasen por el mismo camino de render. Si una forma no tiene normales o material iluminado, `Shape::draw` usa un shader simple; si tiene normales y `lightingEnabled`, usa el shader Blinn-Phong.

Ciudad, Porsche y cielo

La ciudad está encapsulada en `Objetos::BloqueCiudad`. El patrón usado es cargar un OBJ base una vez y dibujarlo varias veces con matrices distintas mediante `RepeatedModel`. Así se reutilizan modelos de carretera, veredas, bancas, postes, árboles, edificios, semáforo y construcciones. `BloqueCiudad` separa `capa_cerca` y `capa_medio`; `setLayerOffsets` mueve ambas capas en Z.

`PorscheCar` carga el vehículo por piezas: pintura, base, carbono, cristales, motor, rejillas, interior, luces, placa, parabrisas y ruedas. Las piezas se agrupan en `opaqueGroup`, `wheelGroup` y `glassGroup`. Las partes opacas se dibujan primero; el parabrisas se dibuja después con `enableAlphaBlending` y `glDepthMask(GL_FALSE)` para conservar transparencia.

`Sky` genera un domo semiesférico con vértices procedurales y carga `Sky_Synthwave_Stars.png`. El cielo se dibuja sin depth test y sin escribir profundidad. Luego se dibuja el sol como esfera emisiva y un `GlowHalo` con blending aditivo. El shader del cielo mezcla la textura original con una versión desplazada en U cerca de la unión, reduciendo la costura visual del mapa panorámico.

Elemento visible	Implementación relevante
Carretera en movimiento	<code>setLayerOffsets</code> desplaza <code>capa_cerca</code> ; el reinicio del offset evita que el camino termine visualmente.
Profundidad urbana	<code>capa_media</code> avanza más lento que <code>capa_cerca</code> , generando parallax.
Semáforo funcional	<code>TrafficLigthObjects</code> dibuja el estado visible y <code>main.cpp</code> usa ese ciclo para frenar el vehículo.
Ruedas animadas	<code>PorscheCar::update</code> acumula rotación proporcional a velocidad y <code>updateWheelTransforms</code> aplica matrices independientes.
Vidrios y halos	Se renderizan con blending y <code>glDepthMask(GL_FALSE)</code> para conservar transparencia o brillo.

La siguiente relación resume cómo cada funcionalidad visible se conecta con una decisión de programación:

Funcionalidad	Archivo principal	Detalle aplicado
Cambio de cámara	<code>CameraSystem.h</code>	<code>pressedOnce</code> evita múltiples cambios por una misma pulsación y <code>activeViewMatrix</code> devuelve la matriz correcta para cada modo.
Movimiento libre	<code>camera.h</code> y <code>CameraSystem.h</code>	<code>Camera::move</code> usa <code>front</code> , <code>right</code> y <code>worldUp</code> ; el mouse actualiza yaw/pitch y recalcula los vectores de cámara.
Carga de ciudad	<code>objetos.h</code>	Cada clase carga un OBJ base, asigna material y guarda matrices de instancia para dibujar varias copias.
Texturizado	<code>draw2D.h</code>	<code>loadTexture2D</code> crea texturas OpenGL, configura filtros y enlaza mapas difusos al material.
Iluminación urbana	<code>lighting.h</code> y <code>objetos.h</code>	Los postes registran spotlights reales y además dibujan haces con <code>GlowCone</code> .
Cielo y sol	<code>Sky.h</code>	El domo se genera proceduralmente, se texturiza con un shader propio y el sol usa material emisivo más halo aditivo.
Animación del Porsche	<code>Porsche.h</code>	<code>updateWheelTransforms</code> actualiza matrices independientes para las cuatro ruedas.
Frenado ante semáforo	<code>main.cpp</code>	El ciclo del semáforo y la distancia calculada modifican <code>speedMultiplier</code> , afectando avance de capas y ruedas.

Esta trazabilidad muestra que la aplicación no depende de una sola rutina grande. La interacción, la carga de datos, el render, la iluminación y la animación están repartidos en módulos específicos. Esto hizo más simple probar cada parte: primero se verificó que los OBJ cargaran, luego que recibieran ma-

teriales, después que respondieran a luces y finalmente que sus matrices se actualizaran correctamente en el loop.

4 Problemas encontrados en la implementación

El primer problema fue adaptar modelos exportados desde Blender al sistema de coordenadas usado por OpenGL. La solución fue centralizar la conversión en `makeTransform`, evitando repetir manualmente cambios de eje y signo en cada objeto.

Otro problema fue la separación entre modelos OBJ y materiales. Aunque los OBJ incluyen archivos MTL, el cargador implementado solo conserva geometría, normales y UV. Por ello se decidió asignar materiales desde C++. Esto permitió controlar mejor el estilo cyberpunk, pero exige definir manualmente textura, brillo, especularidad y alfa por grupo de objetos.

La transparencia también requirió cuidado. Cristales, halos, haces de luz, ventanas y semáforo no podían dibujarse igual que objetos opacos. La solución fue usar blending y desactivar temporalmente la escritura al depth buffer con `glDepthMask(GL_FALSE)`. El cielo también se dibuja sin depth test para comportarse como fondo.

El límite `MAX_LIGHTS = 12` condicionó la iluminación. La escena usa dos luces direccionales y ocho spotlights de postes, por lo que queda dentro del límite. Para no saturar el shader, las ventanas iluminadas se implementaron como superficies aditivas y no como luces reales.

Finalmente, la animación de carretera infinita requería continuidad visual. En vez de mover el auto por el mundo, se movieron capas de ciudad alrededor del Porsche. Esto simplificó la cámara y permitió controlar el frenado ante el semáforo con un `speedMultiplier` interpolado.

Problema	Causa	Solución aplicada
Modelos con escalas y ejes distintos	Exportación desde Blender con convención diferente a la usada en OpenGL.	Uso de <code>makeTransform</code> para convertir posición, escala y rotación antes de instanciar objetos.
Luces que debían moverse con la ciudad	Los postes pertenecen a una capa animada y su posición mundial cambia cada frame.	Se llama <code>clearLights</code> y luego <code>addSpotLight</code> usando matrices actualizadas.
Cristales y halos con errores de profundidad	La transparencia no debe escribir al depth buffer como un objeto opaco.	Se usa blending y <code>glDepthMask(GL_FALSE)</code> durante el dibujo transparente o aditivo.
Carretera infinita sin desplazar el vehículo	Mover el auto complicaba cámara, frenado y referencias visuales.	Se mueve la ciudad por capas y se mantiene el Porsche como referencia principal.
Semáforo visual y comportamiento del auto	La luz dibujada y la lógica de frenado debían coincidir en tiempo.	Se usa el mismo <code>animationTime</code> para el estado visible y para calcular <code>speedMultiplier</code> .

5 Conclusiones

Cyberpunk Velocity integra en una misma escena transformaciones 3D, cámara perspectiva, carga de mallas, texturizado, materiales, iluminación, blending, niebla y animación temporal. La arquitectura por cabeceras permitió separar responsabilidades: `main.cpp` coordina el loop, `CameraSystem.h` controla vistas e input, `objetos.h` construye la ciudad, `Porsche.h` encapsula el vehículo y `Sky.h` controla el fondo.

El proyecto muestra que una escena compleja puede construirse con componentes relativamente simples si cada pieza tiene una responsabilidad clara. La separación entre luces reales, brillos decorativos, materiales manuales y capas animadas hizo posible controlar el resultado visual sin depender de un motor externo completo.

6 Próximos pasos

Como mejora futura se podría implementar instancing por GPU con `glDrawElementsInstanced` para dibujar muchas copias de edificios, árboles o segmentos de carretera con menor costo. También sería útil leer archivos MTL para importar materiales básicos automáticamente y luego permitir ajustes manuales encima.

Otro paso sería agregar faros funcionales al Porsche, más efectos de postprocesamiento, sombras proyectadas, control de exposición o bloom. En interacción, se podrían añadir indicadores en pantalla para cámara activa, estado del semáforo y pausa. Finalmente, se podría organizar la escena en archivos de datos para modificar posiciones y materiales sin recompilar.

7 Referencia

- Khronos Group. *OpenGL 3.3 Core Profile Specification*.
- GLFW. *GLFW Documentation*: creación de ventanas, contexto OpenGL e input.
- GLAD. *OpenGL Loader Generator*: carga de funciones OpenGL.
- stb. *stb_image.h*: carga de imágenes PNG/JPG para texturas.
- Archivos locales del proyecto: `main.cpp`, `draw2D.h`, `draw3D.h`, `lighting.h`, `camera.h`, `CameraSystem.h`, `objetos.h`, `Porsche.h`, `Sky.h`.